# CS714 Mid-semester test – Practice questions & answers

Question 1: Data pre-processing [4 marks]

Describe 2 approaches to deal with missing data in a tabular dataset. For each approach, give an example of a situation in which the approach would be preferred, and why.

(2 marks for each approach – 1 mark for the approach, 1 mark for the example)

Some possible approaches (non-exhaustive):

1) Removing the attribute with missing values. We could use this approach if there were a very large number of missing values in an attribute, therefore it is difficult to input the values and it might not be very useful for the prediction.
2) Imputing the values. Several imputation approaches can be used, including statistical approaches (e.g., replacing continuous value by average for this attribute), or machine learning approaches (i.e., a model is learned on the other attributes to predict missing values for the attribute). This last type of imputation could be used if there is a reasonable number of missing values in an attribute, and a pattern can be learned from other attributes to predict them.
3) Collecting more data…
4) Removing the instances with missing values…
5) Etc.

Question 2: Deep Neural Networks and generalisation [4 marks]

For the following situations, describe a simple solution and the intuition behind it:

i. Your model has a high bias after training. (2 marks)

Sign of underfitting; Possible solutions to try: (1 mark for solution, 1 mark for intuition)

1. Increase model size, i.e., add more layers and/or neurons per layers (increase effective capacity of the network).
2. Add more (relevant) data to the training set (not enough data to model patterns).
3. Use a pretrained NN on a relevant task (transfer learning).
4. Train longer (if no overfitting and enough date) (training may be stopped too soon).
5. Etc. (other possible valid solutions)

ii. Your model has a low bias and a high variance after training. (2 marks)

Sign of overfitting. Possible solutions to try: (1 mark for solution, 1 mark for intuition)

1. Add data to the training set (must be relevant to the task) (training data not sufficient to generalise in test set)
2. Use regularisation techniques (model learned is too complex).

3. Etc. (other possible valid solutions)

Question 3: Design choices [6 marks]

You trained a simple neural network with the following settings: 2 hidden layers with ReLU units, 5 neurons per layer, random initialisation, trained for 2000 epochs, with L1 regularisation for each hidden layer. Unfortunately, both your training and validation accuracies are equal to 50%, while you know other models performed up to 95% accuracy on the same problem. For each of the following suggestions, explain if it is likely to improve the validation accuracy and why, if implemented in the next iteration of the training.

- Increasing the size of the neural network, i.e., adding more layers and/or more neurons per layer.
- Initialising the weights to 0.
- Using dropout in each layer.

(2 marks per suggestion - 1 for the answer, 1 for the reasoning)

- Increasing the size of the neural network, i.e., adding more layers and/or more neurons per layer.
  - ➔ It is likely to increase the accuracy, assuming there is enough data to train the additional weights. This is because we are in an underfitting situation (training accuracy is low compared to what can be achieved). One reason might be that the model is not complex enough to model the data. Increasing the size of the network might help to get a better trained model.
- Initialising the weights to 0.
  - ➔ It will not help to increase the accuracy. Initialising the weights to 0 will create symmetry, leading to the network not being able to learn much. It will lead to even worst accuracy.
- Using dropout in each layer.
  - ➔ Dropout is a regularisation technique, helping to prevent overfitting. This is not an overfitting situation; therefore, it is not likely to help increase the accuracy here.

Question 4: Evaluation [3 marks]

Consider the following confusion matrix, resulting from the evaluation of a model on a test set containing 100 instances.

| | | True class | |
|---|---|---|---|
| | | Positive | Negative |
| Predicted class | Positive | 5 | 20 |
| | Negative | 10 | 65 |

Calculate the accuracy and recall. What do you think about this model performance?

1 mark for the accuracy calculation.

1 mark for the recall calculation.

1 mark for the discussion of the performance.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{70}{100} = 0.70\ (70\%)$$

$$Recall = \frac{TP}{TP + FN} = \frac{5}{15} = 0.33\ (33\%)$$

The accuracy is relatively high (70%) but the recall is quite low (33%). This show that even if the model seems to perform relatively well globally, it is not very good at predicting the positive class. This comes from class imbalance, with the positive class being the minority class here. If we care about maximizing the true positives, we should consider that this model does not perform well.

Question 5: Batch normalisation [4 marks]

Explain 2 of the effects batch normalisation has on learning in a Neural Network.

2 marks per effect (1 mark for naming, 1 mark for explanation)

Possible effects:

- Speed up the training by normalising the input of each layer. Bring the input values into a range that allow the gradient not to become too small when using activation functions such as sigmoid or tanh. This allows for a faster training.
- Reduce risk of vanishing/exploding gradients. Similar explanation as previous point.
- Regularisation effect. Because the normalisation is done on mini-batches and not the entire dataset, some noise is introduced in the activation values. This has a regularisation effect as it helps prevent overfitting.
- Reduces the influence of the weight initialisation. Too small weights can lead to slow learning as gradient values are small. Too large weights can lead to saturation or exploding gradients. Using batch regularisation will help control the weight values and therefore limit the effect of weight initialisation.

Question 6: Attention mechanism and Transformers [4 marks]

a) Explain in your own words what is multi-head attention. (2 marks)

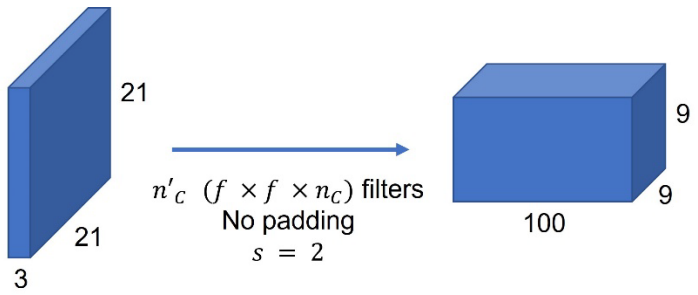
Multi-head attention is a layer in transformers which stacks several attention layers (scaled-dot attention). Each head calculates an attention score reflecting correlations between parts of the input. Weight matrices (Query, Value and Key) for each head are learned during the training process.

b) Explain why multi-head attention is useful for Transformer models. (2 marks)

Multi-head attention is beneficial because it allows the Transformer to focus on different aspects of the input sequence. Using a single attention head would have an averaging effect. Intuitively, the multi-head attention allows the Transformer to spread its attention on different parts of the sequence, instead of averaging it over the full sequence.

Question 7: Convolutional Neural Networks [5 marks]

a) Given the following representation of a convolutional layer with input and output feature maps:




What are the values of the following hyperparameters (explain your workings):

i. The number of filters $n'_C$ (0.5 mark)

$n'_C = 100$ (depth of output feature map)

ii. The filter size $f$ (1 mark)

(n+2p-f)/2 + 1 = 9 → (21+0-f)/2 + 1 = 9 → f = (21 – 16) = 5

iii. The filter width $n_C$ (0.5 mark)

$n_C = 3$ (depth of input feature map)

b) Consider a Convolutional Neural Network trained to classify objects in an image. After training, if you were to visualise the neurons' functions at different depths in the network, which neurons are more likely to behave like edge detectors? Explain your reasoning. (3 marks)

(1 mark for answer, 2 marks for reasoning)

Neurons in layers close to the input are likely to behave as edge detectors as they are building low level representations such as edges. Deeper layers take these low-level representations are combine them to build more complex features.